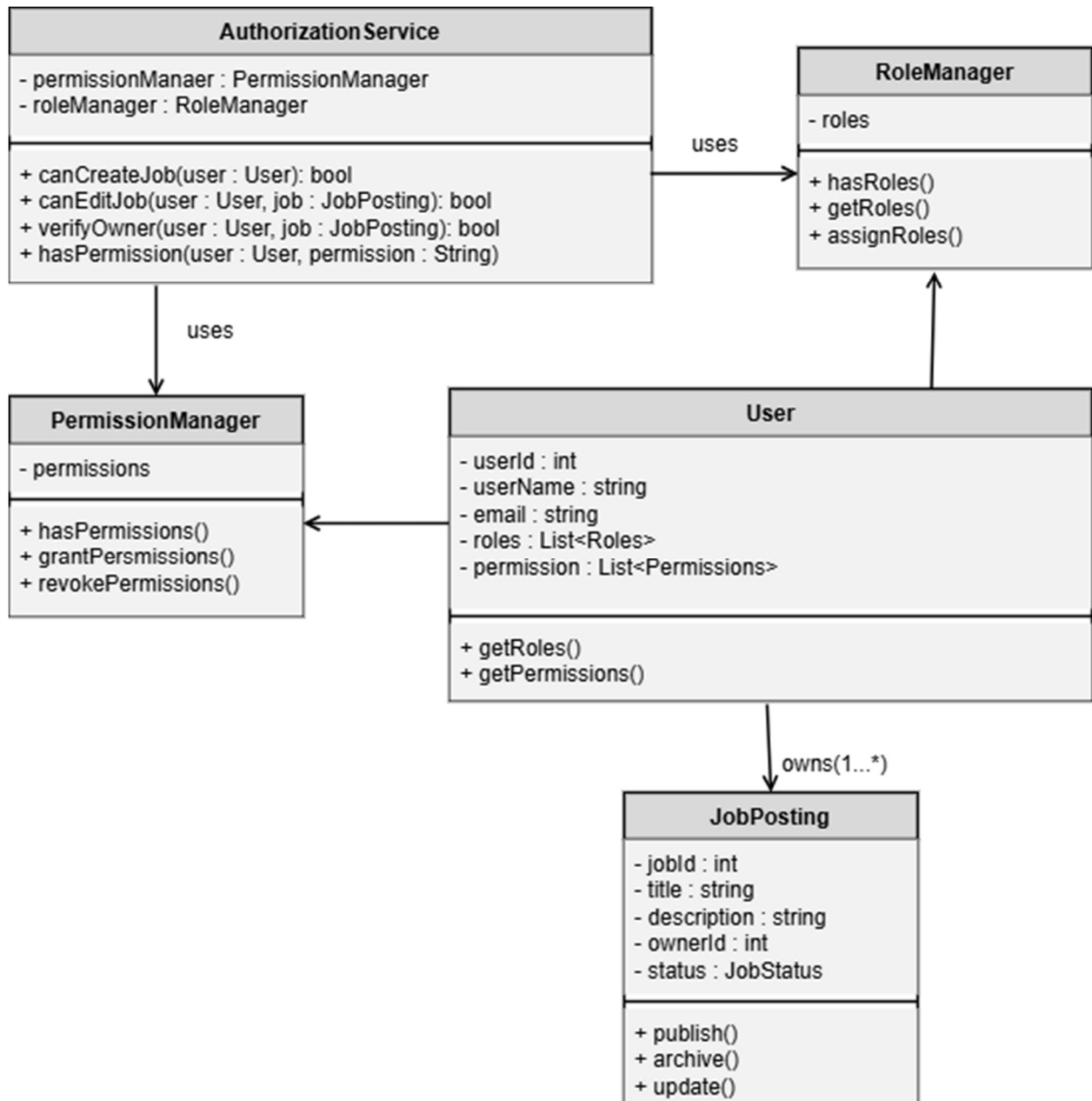


4.0 DATA DESIGN

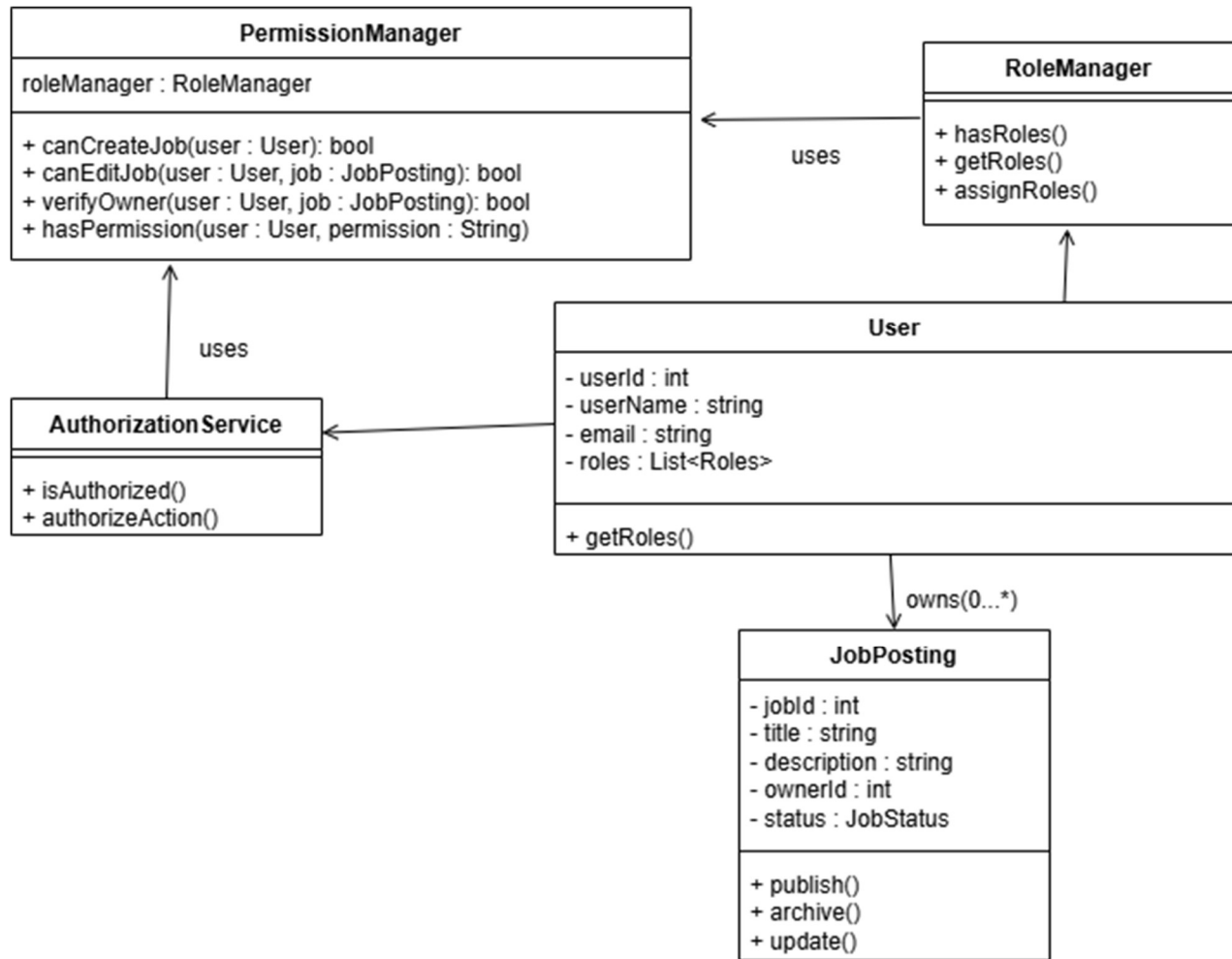
4.1 Class Design

Class Diagrams – 1.8 Job Posting Service

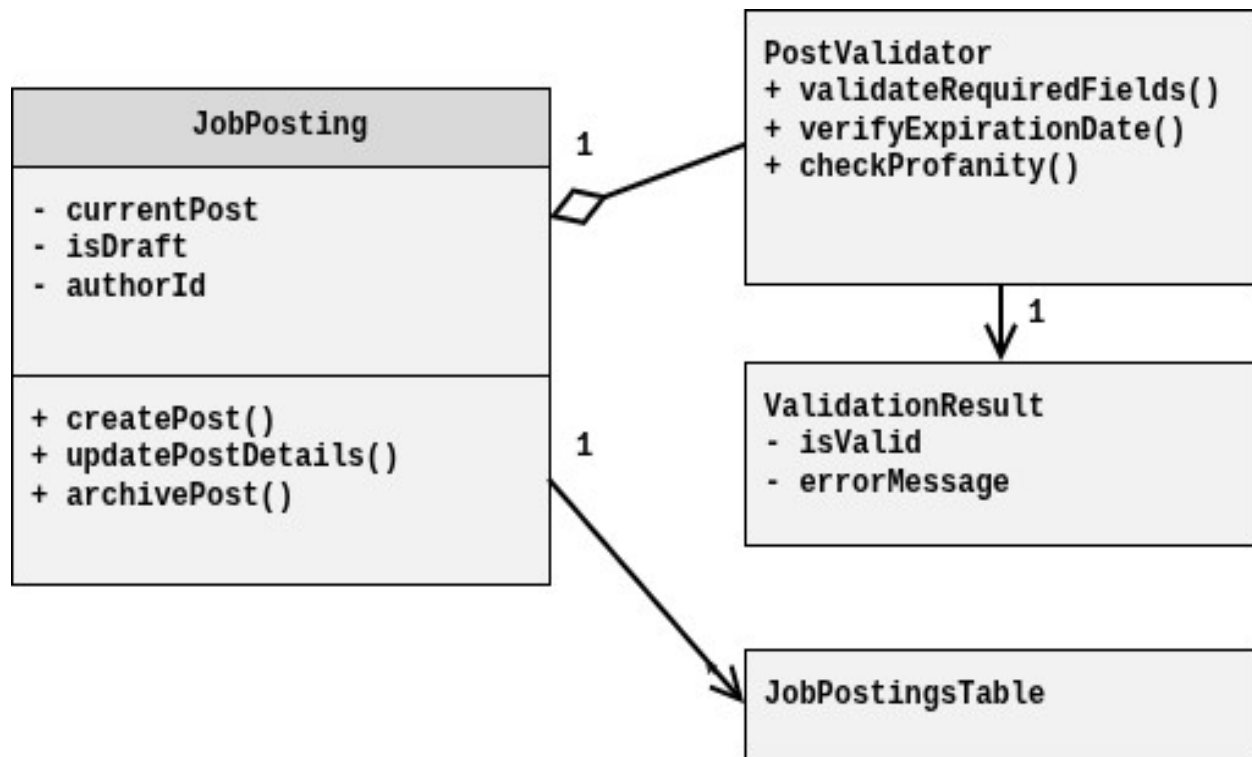
1.8.1-CD AuthorizationService Class Diagram



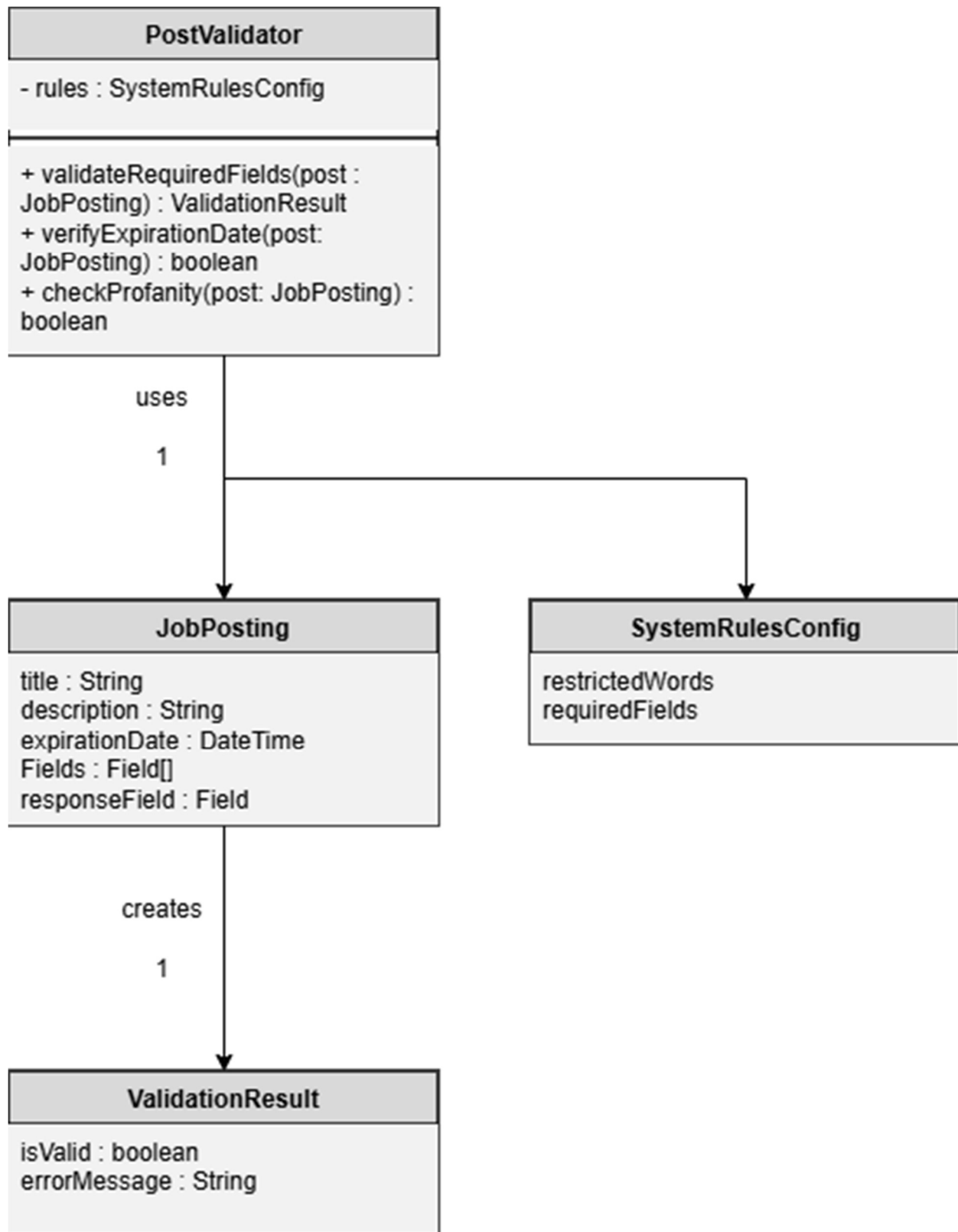
1.8.1-CD PermissionManager Class Diagram



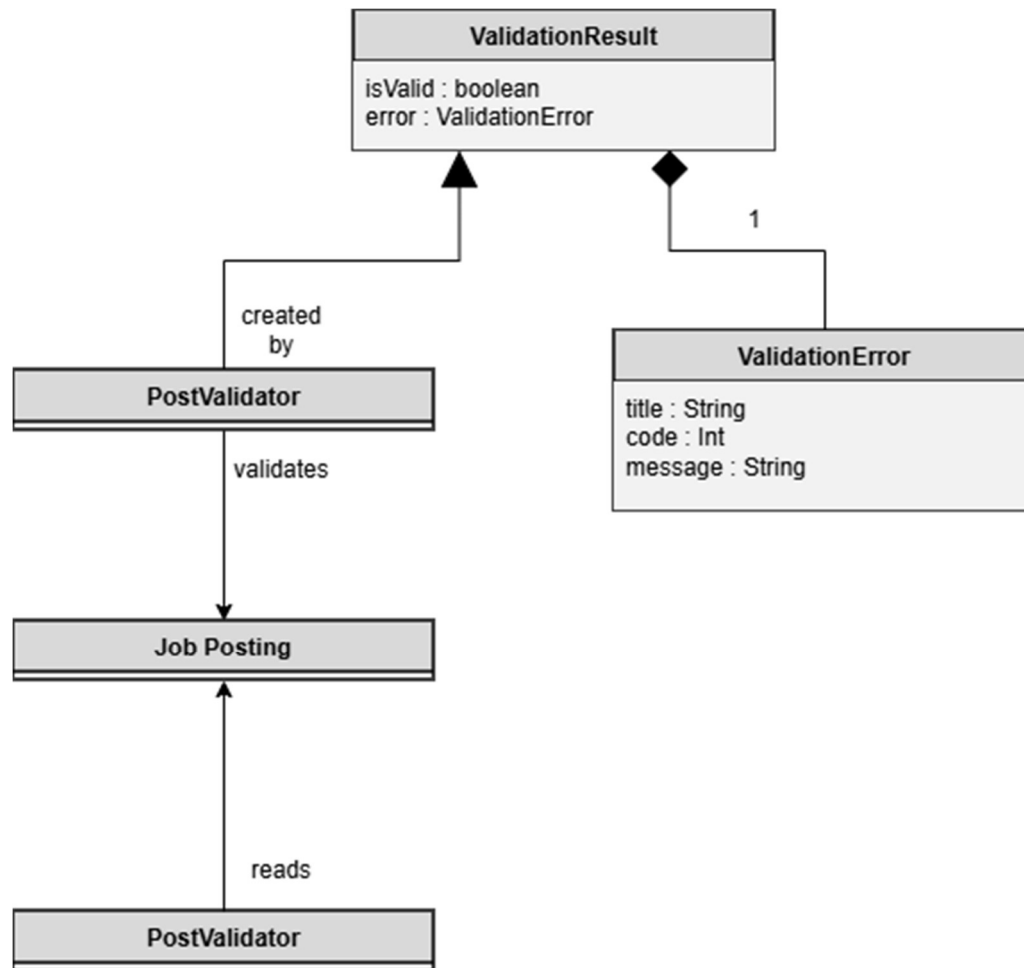
1.8.2-CD JobPosting Class Diagram



1.8.3-CD PostValidator Class Diagram



1.8.3-CD ValidationResult Class Diagram



5.0 COMPONENT DESIGN

1.2 & 1.3 Web and Mobile Client Algorithms

1.2.1 & 1.3.1-ALG updateView() Algorithm

Purpose:

Determines which view should be shown and updates navigation state.

Pseudocode

```
updateView()  
  
  IF selected view is different from currentView THEN  
    SET currentView <- newView  
    DO renderSelectedView()  
  
  END
```

Local Data:

newView : View

currentView : View

1.2.1 & 1.3.1-ALG selectNavigationTarget() Algorithm

Purpose:

Determines the correct target view from a selected menu/button action.

Pseudocode

```
selectNavigationTarget()  
  SET userState <- getCurrentUserState()  
  Set route <- getSelectedRoute()  
  
  IF userState permits access to route THEN  
    SET targetView <- View associated with route  
  ELSE  
    SET targetView <- Default or AccessDenied View  
  END  
  
  RETURN targetView  
END
```

Local Data:

userState : State

route : Route

1.2.2 & 1.3.2-ALG onInit() Algorithm

Purpose:

Performs initial view setup when a page/view loads.

Pseudocode

```
onInit()  
  SET config <- viewConfig  
  DO apply configuration to View  
  SET data <- Load ViewData  
  View.setData(data)  
  View.render()  
END
```

Local Data:

config : Config

data : ViewData

1.2.2 & 1.3.2-ALG renderSelectedView() Algorithm

Purpose:

Coordinates rendering after navigation or state change.

Pseudocode

```
renderSelectedView(selectedView)  
  
  GET selectedView  
  
  IF selectedView is empty  
    PUT "No view has been selected to render"  
    RETURN false  
  
  READ viewDefinition for selectedView from the view registry  
  
  IF viewDefinition is empty  
    PUT "Selected view could not be found"  
    RETURN false  
  
  onInit(viewDefinition)  
  render(viewDefinition)  
  updateAccessibility(viewDefinition)  
  
  RETURN true
```

Local Data:

selectedView : View

1.2.2 & 1.3.2-ALG render() Algorithm

Purpose:

Displays view content and coordinates rendered UI components.

Pseudocode

```
render(view)

  GET view
  READ componentList from view

  IF componentList is empty
    PUT "No content is available to display"
    RETURN false

  FOR each component IN componentList
    IF component.isVisible is true
      PUT component to the view container
    ELSE
      skip to the next component

  apply the active layout rules to the view container
  PUT view to screen

  RETURN true
```

Local Data:

view : View

componentList : List of Component Objects

1.2.3 & 1.3.3-ALG submitForm() Algorithm

Purpose:

Coordinates the full form submission workflow.

Pseudocode

```
submitForm(form)

  GET form
  READ inputValues from the form fields

  IF checkRequiredFields(inputValues) is false
    displayValidationFeedback("Please complete all required fields")
    RETURN false

  validationErrors ← validateInputs(inputValues)

  IF validationErrors is not empty
    displayValidationFeedback(validationErrors)
    RETURN false

  payload ← buildPayload(inputValues)
  response ← sendAPIRequest(payload)

  IF response is successful
    displayValidationFeedback("Your form was submitted successfully")
    RETURN true
  ELSE
    displayValidationFeedback("Submission failed, please try again")
    RETURN false
```

Local Data:

form : HTML DOM Object

inputValues : list of Strings

1.2.3 & 1.3.3-ALG validateInputs() Algorithm

Purpose:

Performs validation across submitted inputs.

Pseudocode

```
validateInputs(formFields)
  isValid ← TRUE
  errors ← empty list

  FOR each field IN formFields
    IF field.type = "text" THEN
      IF field.value contains special characters THEN
        ADD "Invalid characters in " + field.name TO errors
        isValid ← FALSE
    IF field.type = "email" THEN
      IF field.value does not match email format THEN
        ADD "Invalid email format" TO errors
        isValid ← FALSE
    IF field.type = "phone" THEN
      IF field.value does not match phone format THEN
        ADD "Invalid phone number format" TO errors
        isValid ← FALSE
    IF field.type = "date" THEN
      IF field.value does not match date format THEN
        ADD "Invalid date format" TO errors
        isValid ← FALSE
    IF field.type = "file" THEN
      IF field.value file extension NOT IN allowedTypes THEN
        ADD "Unsupported file type for " + field.name TO errors
        isValid ← FALSE

  IF isValid = FALSE THEN
    CALL displayValidationFeedback(errors)

  RETURN isValid
END
```

Local Data:

isValid : Boolean

field : HTML DOM Object

1.2.3 & 1.3.3-ALG checkRequiredFields() Algorithm

Purpose:

Checks whether mandatory fields are completed.

Pseudocode

```
checkRequiredFields(formFields)
  isComplete ← TRUE
  missingFields ← empty list

  FOR each field IN formFields
    IF field.required = TRUE THEN
      IF field.value = NULL OR field.value = "" THEN
        ADD field.name TO missingFields
        isComplete ← FALSE

  IF isComplete = FALSE THEN
    CALL displayValidationFeedback(missingFields)

  RETURN isComplete
END
```

Local Data:

field : HTML DOM Object

isComplete : Boolean

1.2.3 & 1.3.3-ALG buildPayload() Algorithm

Purpose:

Converts form data into a request payload.

Pseudocode

```
buildPayload(formFields)
  payload ← empty object

  FOR each field IN formFields
    IF field.type = "file" THEN
      payload[field.name] ← ENCODE field.value TO base64
    ELSE
      payload[field.name] ← field.value

  payload["timestamp"] ← GET current timestamp
  payload["clientType"] ← GET client type (web or mobile)

  RETURN payload
END
```

Local Data:

field : HTML DOM Object

1.2.3 & 1.3.3-ALG displayValidationFeedback() Algorithm**Purpose:**

Displays errors or confirmation messages to the user.

Pseudocode

```
Function displayValidationFeedback()  
  
    validationResults ← getValidationResults()  
  
    IF validationResults contains errors THEN  
  
        formattedMessages ← formatErrorMessages(validationResults)  
  
        updateFormState(formattedMessages)  
  
    ELSE  
  
        updateFormState("Validation Successful")  
  
End
```

Local Data:

formattedMessages : List of Strings

1.2.4 & 1.3.4-ALG updateClientState() Algorithm**Purpose:**

Updates stored client state after layout/accessibility changes.

Pseudocode

```
Function updateClientState(clientUpdate)  
  
    screenState ← onUpdate(clientUpdate)  
  
    layoutState ← dynamicUI(screenState)  
  
    accessibilityState ← updateAccessibility(layoutState)  
  
    Store updated client state  
  
End
```


Local Data:

userId : Int
newSettings : Settings
settings : Settings
isValid : Boolean
currentDateTime : DateTime
updateResult : Boolean
errorMessage : String

1.2.4 & 1.3.4-ALG DynamicUI () Algorithm**Purpose:**

Determines whether web or mobile layout rules should apply.

Pseudocode

```
METHOD DynamicUI ()  
  
    GET current State  
  
    READ width  
    READ height  
  
    IF width < MobileThreshold THEN  
        layout ~ "Mobile"  
    ELSE  
        layout ~ "Desktop"  
    ENDIF  
  
    PUT layout INTO current State  
  
    updateAccessibility ()  
  
    BackgroundHandler ()  
  
    RETURN layout  
  
END METHOD
```

Local Data:

currentState : State
width : int
height : int

1.2.4 & 1.3.4-ALG updateAccessibility() Algorithm

Purpose:

Applies accessibility preferences to the client UI.

Pseudocode

```
updateAccessibility (userPreferences)

  IF not userPreference
    RETURN false

  IF userPreference.fontSize
    set_global_font_size(userPreference.fontSize)

  IF userPreferences.highContrastMode
    enable_css_theme("high-contrast-mode")
  ELSE
    disable_css_theme("high-contrast-mode")

  IF userPreferences.screenReaderOptimization
    set_aria_attributes(active, true)

  RETURN true

END
```

Local Data:

userPreferences : Config

1.2.4 & 1.3.4-ALG sendAPIRequest() Algorithm

Purpose:

Coordinates request creation, sending, response parsing, and error handling.

Pseudocode

```
sendAPIRequest(endpointUrl, httpMethod, requestPayload, sessionToken)
  requestHeaders <- {
    "Content-Type": "application/json",
    "Authorization": "Bearer " + sessionToken
  }

  connectionOptions <- {
    "method": httpMethod,
    "headers": requestHeaders,
    "body": requestPayload
  }

  TRY
    response <- AWAIT fetch(endpointUrl, connectionOptions)
    RETURN response
  CATCH error
    RETURN error

END
```

Local Data:

requestHeaders : Dictionary

connectionOptions : Dictionary

1.2.4 & 1.3.4-ALG ResponseParser() Algorithm

Purpose:

Interprets API responses and extracts usable data..

Pseudocode

```
responseParse(response)
  IF not response
    RETURN {"status": "error", "message": "Null response"}

  TRY
    status <- response.status

    IF status >= 500
      RETURN {
        "status": "server_error",
        "message": "Remote host failed. Code: " + status
      }
    ELSE IF status >= 400
      RETURN {
        "status": "client_error",
        "message": "Invalid input from client. Code: " + status
      }
    ELSE
      data <- response.parse()
      RETURN {
        "status": "success",
        "data": data
      }
  CATCH error
    RETURN {
      "status": "parse_error",
      "message": "Failed to parse body"
    }

END
```

Local Data:

status : int

1.4 API Gateway / Orchestrator Algorithms

1.4.1-ALG authenticateIfNeeded() Algorithm

Purpose:

Authenticate user requests if necessary and update RequestContext with the result.

Pseudocode

```
authenticateIfNeeded(requestContext):  
  
    // Step 1: Determine if authentication is required  
    IF requestContext.path IN publicEndpoints  
        RETURN AuthenticationResult(success = TRUE, userId = "anonymous", roles = [])  
  
    // Step 2: Extract token from request headers  
    authenticationToken ← requestContext.headers["Authorization"]  
  
    IF authenticationToken == NULL  
        RETURN AuthenticationResult(success = FALSE, userId = NULL, roles = [])  
  
    // Step 3: Build new AuthenticationRequest  
    authenticationRequest ← NEW AuthenticationRequest  
    authenticationRequest.token ← authenticationToken  
  
    // Step 4: Send to AuthenticationHandler  
    authenticationResult ← AuthenticationHandler.validateCredentials(authRequest)  
  
    // Step 5: Update RequestContext if successful  
    IF authenticationResult.success == TRUE  
        requestContext.userId ← authenticationResult.userId  
        requestContext.roles ← authenticationResult.roles  
        RETURN authenticationResult  
    ELSE  
        RETURN authenticationResult
```

Local Data:

authenticationRequest : AuthenticationRequest

authenticationResult : AuthenticationRequest

1.4.2-ALG applyRoutingRules() Algorithm

Purpose:

Apply routing rules and definitions to a user request before it is routed to its intended destination.

Pseudocode

```
applyRoutingRules(routerRequest):  
  
    // Step 1: Load routing table from Database  
    routingRules ← routingTable.rules  
    routeDefinitions ← routingTable.routeDefinitions  
  
    // Step 2: Evaluate each rule against routerRequest  
    FOR EACH rule IN routingRules  
  
        match ← rule.evaluate(routerRequest)  
  
        IF match == TRUE  
            // Step 3: Find corresponding route definition in Database  
            FOR EACH routeDef IN routeDefinitions  
                IF routeDef.matches(routerRequest)  
                    RETURN routeDef  
  
    // Step 4: Return error message if no rules are matched  
    RETURN Error("No matching route definition found")
```

Local Data:

routingRules : List<RoutingRule>

routingDefinitions : List<RoutingDefinition>

1.4.3-ALG validateCredentials() Algorithm

Purpose:

Authenticate submitted user credentials against entries stored in the authentication database.

Pseudocode

```
validateCredentials(authenticationRequest):  
  
    // Step 1: Retrieve user credential record  
    userCred ← authDB.getUser(authenticationRequest.username)  
  
    IF userCred == NULL  
        RETURN AuthenticationResult(success = FALSE, userId = NULL, roles = [])  
  
    // Step 2: Check account lockout status  
    IF userCred.failedAttempts >= MAX_FAILED_ATTEMPTS  
  
        RETURN AuthenticationResult(success = FALSE, userId = NULL, roles = [])  
  
    // Step 3: Hash the provided password  
    hashedInput ← HASH(authenticationRequest.password, userCred.salt)  
  
    // Step 4: Compare hashes  
    IF hashedInput != userCred.passwordHash  
        userCred.failedAttempts ← userCred.failedAttempts + 1  
        authDB.updateUser(userCred)  
  
        RETURN AuthenticationResult(success = FALSE, userId = NULL, roles = [])  
  
    // Step 5: Successful authentication  
    userCred.failedAttempts ← 0  
    authDB.updateUser(userCred)  
  
    roles ← authDB.getRoles(userCred.username)  
  
    RETURN AuthenticationResult(success = TRUE, userId = userCred.username, roles =  
    roles)
```

Local Data:

userCred : AuthenticationRequest

hashedInput : Hash<AuthenticationRequest>

1.4.4-ALG validate() Algorithm

Purpose:

Validates an incoming request by cleaning the request body, retrieving route rules, verifying the user, checking the request requirements, and returning either a valid request result or a validation error.

Pseudocode

```
validate(request)
  GET body from request
  GET route from request
  GET token from request

  cleanData ← cleanRequestBody(body)
  rules ← readRulesForRoute(route)
  user ← verifyUserToken(token)

  checkResult ← check(cleanData, rules, user, route)

  IF checkResult is not valid THEN
    error ← createValidationError(checkResult)
    result ← createInvalidRequestResult(error)
    RETURN result

  validRequest ← createValidatedRequest(cleanData, user, route)
  result ← createValidRequestResult(validRequest)

  RETURN result
```

Local Data:

cleanData: Sanitized Request Data

rules: Validation Rules

user: User

checkResult: Check Result

error: Validation Error

validRequest: Valid Request

result: Validation Result

1.4.4-ALG check() Algorithm

Purpose:

Checks request data against user authentication, route permissions, required fields, and validation rules to determine whether the request is valid.

Pseudocode

```
check(data, rules, user, route)
  IF user is not verified THEN
    checkResult ← createFailedCheckResult("Invalid user")
    RETURN checkResult
  permissionStatus ← checkRoutePermission(user, route)
  IF permissionStatus is not allowed THEN
    checkResult ← createFailedCheckResult("Permission denied")
    RETURN checkResult
  requiredFieldStatus ← checkRequiredFields(data, rules)
  IF requiredFieldStatus has missing fields THEN
    checkResult ← createFailedCheckResult("Missing required fields")
    RETURN checkResult
  ruleStatus ← compareDataToRules(data, rules)
  IF ruleStatus is not valid THEN
    checkResult ← createFailedCheckResult("Invalid request data")
    RETURN checkResult
  checkResult ← createPassedCheckResult("Request passed validation")
  RETURN checkResult
```

Local Data:

permissionStatus: Permission Status

requiredFieldStatus: Required Field Status

ruleStatus: Rule Status

checkResult: Check Result

1.4.4-ALG run() Algorithm

Purpose:

Runs the service request by finding the correct service route, returning an error if the route is missing, and building a response from the service result.

Pseudocode

```
METHOD run(request)
  GET route from request
  serviceRoute ← readServiceRoute(route)

  IF serviceRoute is not found THEN
    response ← createErrorResponse(404, "Service route not found")
    RETURN response

  serviceResult ← runServiceRoute(serviceRoute, request)
  response ← buildResponse(serviceResult)
  RETURN response
```

Local Data:

serviceRoute: Route Definition

serviceResult: Service Result

response: Response

1.5 Authentication Service Algorithms

1.5.1-ALG registerUser() Algorithm

Purpose:

The algorithm that takes the user input, puts it in a package, and sends it off to the corresponding database. Note: this algorithm DOES NOT verify the data.

Pseudocode

```
registerUser()

  GET userEmail
  GET userPassword
  GET userType
  userInfoPackage ← {userEmail, userPassword, userType}

  IF userType = STUDENT
    WRITE userInfoPackage -> studentDB

  ELSE userType = BUSINESS
    WRITE userIngoPackage -> businessDB

END
```

Local Data:

userEmail : string

userPassword : string

userType : enumeration

studentDB : external database

connectionbusinessDB : external database connection

1.5.1-ALG validateEmail() Algorithm**Purpose:**

To ensure that the email is in the correct format and doesn't extend past 254 characters according to the RFC 5321 section 4.5.3.1.3 standards. NOTE: This does not confirm that the email actually exists.

Pseudocode

```
validateEmail(email)

    normalizedEmail <- lower(email)

    IF not normalizedEmail matches emailPattern
        PUT "Email must be in the format user@service.domain"
        RETURN false

    IF length(normalizedEmail) > 254
        PUT "Email is too long"
        RETURN false

    PUT "Email validated successfully"

    RETURN true

END
```

Local Data:

normalizedEmail : String

emailPattern : Regular Expression

1.5.1-ALG validatePassword() Algorithm

Purpose:

To ensure that the password is compliant with NIST SP 800-63B section A password standards.

Pseudocode

```
validatePassword(password)

  IF password length is less than 8
    PUT "Password length must be at least 8 characters long"
    RETURN false

  IF password length is greater than 128
    PUT "Password length must be at most 128 characters long"
    RETURN false

  IF password is in blockedPasswords
    PUT "Password is blocked"
    RETURN false

  PUT "Password was validated successfully"
  RETURN true

END
```

Local Data:

blockedPasswords : List of Strings

1.5.1-ALG hashPassword() Algorithm

Purpose:

Acts as a wrapper for a password hashing function from a certified cryptography library, and returns the hashed password.

Pseudocode

```
hashPassword(password)

  hashedPassword <- Hash function from trusted encryption library

  RETURN hashedPassword

END
```

Local Data:

hashedPassword : String

1.5.2-ALG checkPermissions() Algorithm

Purpose:

This is for any service looking for what kinds of permissions are available to a specific type of user, based off of the user object.

Pseudocode

```
checkPermissions(user)

  type <- user.getUserType()

  IF type = STUDENT
    return permissions <- studentPermissions

  IF ELSE type = school
    return permissions <- schoolPermissions

  ELSE userType = BUSINESS
    return permissions <- businessPermissions

END
```

Local Data:

user : User

1.5.3-ALG LoginUser() Algorithm

Purpose:

To verify a user's credentials and create a login session if the credentials are valid.

Pseudocode

```
LoginUser()

  GET userName
  GET password

  READ userAccount FROM UserDatabase

  IF userAccount does not exist
    PUT "Invalid user name or password"
    RETURN false

  IF userAccount is locked
    PUT "Account is locked"
    RETURN false

  IF password does not match userAccount.password
    PUT "Invalid user name or password"

    IF failedLoginAttempts >= maximumAttempts
      UPDATE userAccount TO locked

    WRITE userAccount TO UserDatabase

    PUT "Invalid user name or password"
    RETURN false

  SET failedLoginAttempts TO 0

  session <- createSession(userAccount)

  WRITE session TO SessionDatabase

  PUT "Login successful"

  RETURN true

END
```

Local Data:

username : String

password : String

userAccount : UserAccount

failedLoginAttempts : Number

maximumAttempts : Number

session : Session

1.5.3-ALG LogoutUser() Algorithm

Purpose:

To end the user's active session and remove access to authenticated features.

Pseudocode

```
LogoutUser()  
  
  GET sessionId  
  
  READ activeSession FROM SessionDatabase  
  
  IF activeSession does not exist  
    PUT "No active session found"  
    RETURN false  
  
  SET activeSession.status <- inactive  
  SET activeSession.logoutTime <- currentTime  
  
  WRITE activeSession TO SessionDatabase  
  
  logActivity(activeSession.userId, "User logged out")  
  
  CLEAR sessionId  
  
  PUT "Logout successful"  
  
  RETURN true  
  
END
```

Local Data:

sessionId : String

activeSession : Session

logoutTime : DateTime

logoutSuccessful : Boolean

1.5.3-ALG CreateSession() Algorithm

Purpose:

To create an active login session for a valid user.

Pseudocode

```
CreateSession(userAccount)

    CREATE sessionId

    SET session.userId <- userAccount.userId
    SET session.role <- userAccount.role
    SET session.createdAt <- currentTime
    SET session.expiresAt <- expirationTime
    SET session.status <- active

    RETURN session

END
```

Local Data:

sessionId : String

userId : Number

role : String

createdAt : DateTime

expiresAt : DateTime

status : String

1.6 Profile Management Service Algorithms

1.6.1-ALG addProfileInformation() Algorithm

Purpose:

Adds a user's profile information by validating the submitted profile details, building a new profile information record, saving it to the profile data store, and returning a success or error result.

Pseudocode

```
addProfileInformation(userId, profileInformation)

  GET userId
  GET profileInformation

  READ existingProfile FROM profile data store
    WHERE existingProfileInformation.userId = userId

  IF existingProfileInformation exists
    PUT "Profile information already exists"
    RETURN false

  SET isValid <- validateProfileInformation(profileInformation)

  IF isValid is false
    PUT "Profile information does not meet program standards"
    RETURN false

  SET newProfileInformation.userId <- userId
  SET newProfileInformation.firstName <- profileInformation.firstName
  SET newProfileInformation.lastName <- profileInformation.lastName
  SET newProfileInformation.headline <- profileInformation.headline
  SET newProfileInformation.bio <- profileInformation.bio
  SET newProfileInformation.location <- profileInformation.location
  SET newProfileInformation.accolades <- profileInformation.accolades
  SET newProfileInformation.skills <- profileInformation.skills
  SET newProfileInformation.education <- profileInformation.education
  SET newProfileInformation.dateAdded <- currentDateTime

  WRITE newProfileInformation to profile data store

  IF profile information was added successfully
    PUT "Profile information added successfully"
    RETURN true
  ELSE
    PUT "Profile information could not be added"
    RETURN false

END
```

Local Data:

userId : Number

updatedProfileInformation : ProfileInformation

existingProfile : Profile
isValid: Boolean
currentDateTime : DateTime
updateResult : Boolean
errorMessage : String

1.6.1-ALG updateProfileInformation() Algorithm

Purpose:

Processes submitted profile updates by reading and parsing each profile information entry, saving the updated data to the profile data store, and returning a failure result if an update is unsuccessful.

Pseudocode

```
updateProfileInformation(profileInfo)

  WHILE i < profileInfo.rows
    updateInfo <- READ profileInfo[i]
    updateString <- parseDocument(updateInfo)
    WRITE updateString[3..*] to profile data store on updateString[0..2]

    IF updateString upload not successful
      PUT "Error happened when saving updates"
      RETURN false

    i <- i + 1

  END
```

Local Data:

profileInfo : ProfileInformation
updateInfo : ProfileInformationEntry
updateString : String
profileDataStore : DataStore
saveSuccessful : Boolean

1.6.1-ALG addCompanyLocation() Algorithm

Purpose:

Adds a company location to the profile data store by validating the submitted address information, saving each location field, and returning a success or error result.

Pseudocode

```
addCompanyLocation(address, city, state, postalCode, country)

    validAddress <- validateAddress(address, city, state, postalCode, country)

    IF validAddress.isValid = false
        PUT validAddress.errorMessages
        RETURN false

    WRITE address to profile data store
    WRITE city to profile data store
    WRITE state to profile data store
    WRITE postalCode to profile data store
    WRITE country to profile data store

    IF address upload is successful
        PUT "Company location uploaded successfully"
        RETURN true

    IF companyLocation upload not successful
        PUT "Company location could not be processed"
        RETURN false

END
```

Local Data:

address : String

city : String

state : String

postalCode : String

Country : String

validAddress : ValidationResult

1.6.2-ALG updateImage() Algorithm

Purpose:

Updates an existing image by locating the current image file, validating and processing the new image file, replacing the stored image details, saving the updated image file, and returning a success or error result.

Pseudocode

```
updateImage ()

    GET userId

    GET imageId

    GET newImageFile

    READ existingImage from image data store

    IF not existingImage
        PUT "Image could not be found"
        RETURN false

    IF no newImageFile
        PUT "No new image file was provided"
        RETURN false

    SET processedImage <- processImage(newImageFile)

    IF processedImage is invalid
        PUT "Image could not be processed"
        RETURN false

    SET existingImage.imageName <- processedImage.imageName
    SET existingImage.imagePath <- processedImage.imagePath
    SET existingImage.imageType <- processedImage.imageType
    SET existingImage.imageSize <- processedImage.imageSize
    SET existingImage.imageWidth <- processedImage.imageWidth
    SET existingImage.imageHeight <- processedImage.imageHeight
    SET existingImage.lastUpdatedDate <- currentDateTime

    Write existingImage to image data store

    IF image update is successful
        PUT "Image updated successfully"
        RETURN true

    IF image update not successful
        PUT "Image could not be updated"
        RETURN false

END
```

Local Data:

existingImage : Image

processedImage : Image

updateResult : UpdateResult
errorMessage : String

1.6.2-ALG processImage() Algorithm

Purpose:

Processes an uploaded image by checking if it is valid, resizing and cropping it to the required dimensions, compressing it to reduce file size, and returning the processed image for storage.

Pseudocode

```
processImage()  
  
  GET imageFile  
  
  IF not imageFile  
    PUT "No image file was provided"  
    RETURN false  
  
  SET isValid <- validateImage(imageFile)  
  
  IF isValid = false  
    PUT "Image file does not meet program standards"  
    RETURN false  
  
  SET resizedImage <- resizeImage(imageFile)  
  
  SET croppedImage <- cropImage(resizedImage)  
  
  SET compressedImage <- compressImage(croppedImage)  
  
  SET processedImage.imageName <- compressedImage.imageName  
  SET processedImage.imagePath <- compressedImage.imagePath  
  SET processedImage.imageType <- compressedImage.imageType  
  SET processedImage.imageSize <- compressedImage.imageSize  
  SET processedImage.imageWidth <- compressedImage.imageWidth  
  SET processedImage.imageHeight <- compressedImage.imageHeight  
  
  RETURN processedImage  
  
END
```

Local Data:

imageFile : File
isValid : Boolean
resizedImage : Image
croppedImage : Image
compressedImage : Image
processedImage : Image

1.6.4-ALG updateSettings() Algorithm

Purpose:

Update a user's settings by getting the current settings record, validating the requested changes, applying the validated updates, saving the updated settings, and returning a success or error result.

Pseudocode

```
updateSettings(userId, newSettings)

  GET userId

  GET newSettings

  IF settings.userId is userId
    READ settings from settings data store
    RETURN true

  IF settings.userId is not userId
    PUT "Settings could not be found"
    RETURN false

  SET isValid <- validateSettings(newSettings)

  IF isValid is false
    PUT "Settings do not meet program standards"
    RETURN false

  SET settings.privacySettings <- newSettings.privacySettings
  SET settings.notificationSettings <- newSettings.notificationSettings
  SET settings.securitySettings <- newSettings.securitySettings
  SET settings.displaySettings <- newSettings.displaySettings
  SET settings.communicationSettings <- newSettings.communicationSettings
  SET settings.lastUpdatedDate <- currentDateTime

  WRITE settings to settings data store

  IF settings update successful
    PUT "Settings updated successfully"
    RETURN true

  IF settings update not successful
    PUT "Settings could not be updated"
    RETURN false

END
```

Local Data:

userId : Int
newSettings : Settings
settings : Settings
isValid : Boolean

currentDateTime : DateTime
updateResult : Boolean
errorMessage : String

1.6.5-ALG LogActivity() Algorithm

Purpose:

To create a record of what happened, who did it, and when it happened.

Pseudocode

```
LogActivity(userId, activityType, activityDescription)

    SET timestamp <- currentTime

    GET ipAddress

    SET securityEvent.userId <- userId
    SET securityEvent.activityType <- activityType
    SET securityEvent.activityDescription <- activityDescription
    SET securityEvent.timestamp <- timestamp
    SET securityEvent.ipAddress <- ipAddress

    WRITE securityEvent to AuditLogDatabase

    IF write fails
        RETURN false

    RETURN true

END
```

Local Data:

userId : Number
activityType : String
activityDescription : String
timestamp : DateTime
ipAddress : String
securityEvent : SecurityEvent
auditLog : AuditLog
logSuccessful : Boolean

1.7 Resume Service Algorithms

1.7.1-ALG createResume() Algorithm

Purpose:

Creates a resume from the student's completed resume form after verifying that all mandatory fields are present.

Pseudocode

```
createResume(resumeFormData, studentId)

GET resumeFormData
GET studentId

validationErrors ← validateRequiredFields(resumeFormData)

IF validationErrors is not empty
  PUT validationErrors INTO resumeResponse
  RETURN resumeResponse
END IF

resumeData ← createResumeData(resumeFormData, studentId)
PUT resumeData INTO resumeSubmission

RETURN resumeData
END
```


1.7.1-ALG uploadResume() Algorithm

Purpose:

Submits a validated student resume to Resume Storage and returns the result of the upload request.

Pseudocode

```
uploadResume(resumeData, studentId)

GET resumeData
GET studentId

validationErrors ← validateResumeData(resumeData)

IF validationErrors is not empty
  PUT validationErrors INTO uploadResponse
  RETURN uploadResponse
END IF

uploadRequest ← createResumeUploadRequest(resumeData, studentId)
SEND uploadRequest TO Resume Storage
uploadResponse ← RECEIVE resumeUploadResponse

IF uploadResponse status is REJECTED
  RETURN uploadResponse
END IF

RETURN uploadResponse
END
```

1.7.1-ALG validateResumeFile() Algorithm

Purpose:

Validates an uploaded resume file by checking that a file was provided, that its file type is allowed, and that its size does not exceed the permitted limit.

Pseudocode

```
validateResumeFile(uploadedFile)

GET uploadedFile
validationErrors ← createEmptyValidationErrorList()

IF uploadedFile is empty THEN
    PUT "A resume file is required." INTO validationErrors
    RETURN createValidationResult(validationErrors)
END IF

IF uploadedFile type is not a supported document type THEN
    PUT "The resume file type is not supported." INTO validationErrors
END IF

IF uploadedFile size exceeds the maximum allowed size THEN
    PUT "The resume file exceeds the maximum allowed size." INTO validationErrors
END IF

IF validationErrors is not empty THEN
    RETURN createValidationResult(validationErrors)
END IF

RETURN createValidationResult(validationErrors)
END
```

1.7.3-ALG saveResume() Algorithm

Purpose:

Save the information from a resume form as a record in the resume database after successful validation.

Pseudocode

```
saveResume(resumeFormData, studentId)
    validationResult ← ValidateResumeForm(formData)
    IF validationResult.isValid IS FALSE THEN
        RETURN CreateResumeResponse(
            status = "REJECTED",
            errors = validationResult.errors,
            message = "Resume failed validation. See errors."
        )
    END IF
    rawSkills ← formData.skills
    skillTags ← EMPTY LIST
    FOR EACH skillName IN rawSkills DO
        skill ← skillRepository.findOrCreate(normalize(skillName))
        skillTag ← NEW SkillTag(
            skillId = skill.skillId,
            context = formData.contextMap[skillName] OR NULL
        )
        APPEND skillTag TO skillTags
    END FOR
    resumeContent ← NEW ResumeContent(
        summary      = formData.summary,
        education     = formData.educationEntries,
        experience    = formData.experienceEntries,
    )
    resume ← NEW Resume(
        studentId     = studentId,
        resumeContent = resumeContent,
        skillTags     = skillTags,
        isDefault     = formData.isDefault OR FALSE,
        createdAt     = currentTimestamp()
    )
    IF resume.isDefault IS TRUE THEN
        resumeRepository.clearDefaultForStudent(studentId)
    END IF
    savedResume ← resumeRepository.save(resume)
    IF savedResume IS NULL THEN
        RETURN CreateResumeResponse(
            status = "REJECTED",
            message = "Database error. Resume could not be saved."
        )
    END IF
    RETURN SuccessResponse()
```

1.7.3-ALG saveSkill() Algorithm

Purpose:

Tag a skill to a student's resume after verifying that it is in the database and create the skill if it does not exist yet.

Pseudocode

```
saveSkill(skillName, context, resumeId):
    normalizedName ← skillName.trim().toLowerCase()
    skill ← skillRepository.findByName(normalizedName)
    IF skill IS NULL THEN
        skill ← NEW Skill(name = normalizedName)
        skill ← skillRepository.save(skill)
        IF skill IS NULL THEN
            RETURN SkillTagResponse(
                status      = "REJECTED",
                message     = "Failed to create skill. Please try again."
            )
        END IF
    END IF
    skillTag ← New SkillTag(skillId = skill.skillId)
    resume.skillTags.append(skillTag)
```

1.7.5-ALG checkResume() Algorithm

Purpose:

Validate a resume submission before storing to the Resume data store.

Pseudocode

```
checkResume(resume)
  errors ← EMPTY LIST

  summaryResult ← checkResumeSummary(resume.summary)
  IF summaryResult IS NOT NULL
    APPEND summaryResult TO errors

  degreeList[] ← resume.degree.toList
  FOREACH degree in degreeList[]
    degreeResult ← checkDegree(degree)
    IF degreeResult IS NOT NULL
      APPEND degreeResult to errors

  experienceList[] ← resume.experience.toList
  FOREACH experience in experienceList[]
    experienceResult ← checkExperience(experience)
    IF experienceResult IS NOT NULL
      APPEND experienceResult to errors

  isValid ← errors.isEmpty()

  RETURN ValidationResult(
    isValid = isValid,
    errors = errors,
    checkedAt = currentTimestamp()
  )
```

Local Data:

errors : List

summaryResult : Object

degreeResult : Object

experienceResult : Object

ValidationResult : Object

1.7.5-ALG validateResumeInput() Algorithm

Purpose:

Validates student-entered resume form data by checking required fields and verifying that contact, experience, education, date, and GPA information meet the required rules.

Pseudocode

```
validateResumeInput(resumeFormData)

GET resumeFormData
validationErrors ← createEmptyValidationErrorList()

IF required resume fields are missing THEN
    PUT "Complete all required resume fields." INTO validationErrors
END IF

IF resumeFormData.email is not empty AND isEmail(resumeFormData.email) is FALSE THEN
    PUT "Enter a valid email address." INTO validationErrors
END IF

IF resumeFormData.phone is not empty AND isPhone(resumeFormData.phone) is FALSE THEN
    PUT "Enter a valid phone number." INTO validationErrors
END IF

FOR EACH experienceEntry IN resumeFormData.experienceEntries
    IF experienceEntry.position is empty THEN
        PUT "Enter a position title for each experience entry." INTO validationErrors
    END IF
    IF isStartDate(experienceEntry.startDate) is FALSE THEN
        PUT "Enter a valid experience start date." INTO validationErrors
    END IF
    IF experienceEntry.endDate is not empty AND isEndDate(experienceEntry.endDate) is FALSE THEN
        PUT "Enter a valid experience end date." INTO validationErrors
    END IF
END FOR

FOR EACH degreeEntry IN resumeFormData.degreeEntries
    IF degreeEntry.degreeName is empty THEN
        PUT "Enter a degree name for each education entry." INTO validationErrors
    END IF
    IF degreeEntry.graduationDate is not empty AND isGradDate(degreeEntry.graduationDate) is FALSE THEN
        PUT "Enter a valid graduation date." INTO validationErrors
    END IF
    IF degreeEntry.gpa is not empty AND isGPA(degreeEntry.gpa) is FALSE THEN
        PUT "Enter a valid GPA value." INTO validationErrors
    END IF
END FOR

RETURN createValidationResult(validationErrors)
END
```

1.8 Job Posting Service Algorithms

1.8.1-ALG isAuthorized() Algorithm

Purpose:

isAuthorized checks if a user has the overall permission/role required to perform job posting actions like creating or editing.

Pseudocode

```
isAuthorized(userRole, requiredRole)
    authorized ← FALSE

    IF userRole == requiredRole
        authorized ← TRUE
    ENDIF

    PUT authorized
```

1.8.1-ALG verifyOwner() Algorithm

Purpose:

verifyOwner checks if the current user ID matches the owner ID of the specific job listing being modified.

Pseudocode

```
verifyOwner(currentUserId, jobListingId)
    READ jobOwnerId FROM database WHERE listingId == jobListingId
    isOwner ← FALSE

    IF currentUserId == jobOwnerId
        isOwner ← TRUE
    ENDIF

    PUT isOwner
```

1.8.2-ALG createPost() Algorithm

Purpose:

The algorithm validates the data, checks if such a record already exists and if data are valid and such a record does not exist then it writes the data to the database as a new post record.

Pseudocode

```
createPost(companyName, jobTitle, requiredSkills, jobDescription, hours,  
           positionType, compensation, location, contactInformation, linksToApply)  
  
  IF data are not valid  
    RETURN error  
  
  IF companyName and jobTitle already exist in database  
    RETURN job post already exists  
  
  IF WRITE is successful  
    RETURN success  
  
  ELSE  
    RETURN error  
  
END
```

1.8.2-ALG modifyPost() Algorithm

Purpose:

The algorithm validates the data and if data are valid then it updates the record that has the same id.

Pseudocode

```
updatePost(postId, companyName, jobTitle, requiredSkills, jobDescription, hours,  
           positionType, compensation, location, contactInformation, linksToApply)  
  
  IF data are not valid  
    RETURN error  
  
  IF WRITE is successful  
    RETURN success  
  
  ELSE  
    RETURN error  
  
END
```


1.8.3-ALG validate() Algorithm

Purpose:

Validates the entry.

Pseudocode

```
validate()
    valid ← true

    valid ← checkRequiredFields()
    IF valid = true THEN
        valid ← verifyExpirationDate()
    END IF

    IF valid = true THEN
        valid ← checkProfanity()
    END IF

    RETURN valid
END validate
```

1.8.3-ALG checkProfanity() Algorithm

Purpose:

Checks for profanity in the entry.

Pseudocode

```
checkProfanity()
    profaneWords ← getProfanityList()
    textToCheck ← title + description + requirements + benefits

    FOR EACH word IN profaneWords
        IF textToCheck contains word THEN
            RETURN false
        END IF
    END FOR

    RETURN true
END checkProfanity
```

1.9 Search Service Algorithms

1.9.1-ALG submitSearch() Algorithm

Purpose:

Submits the search terms.

Pseudocode

```
submitSearch(searchType, searchQuery, searchFilters)
    searchResult <- EMPTY

    IF searchType == "JOB"
        executeJobSearch(searchQuery, searchFilters)

    IF searchType == "CANDIDATE"
        executeCandidateSearch(searchQuery, searchFilters)

    PUT searchResults

END
```

1.9.1-ALG advancedSearch() Algorithm

Purpose:

The algorithm validates the advanced search data and if data are valid then it performs the search according to the search data either for company search or for job search or for candidate search.

Pseudocode

```
advancedSearch(advancedSearchData)

  IF advancedSearchData for company search
    IF company search data are not valid
      return error
    ELSE
      IF READ database for company search data is successful
        return success
      ELSE
        return error

  IF advancedSearchData for job search
    IF job search data are not valid
      return error
    ELSE
      IF READ database for job search data is successful
        return success
      ELSE
        return error

  IF advancedSearchData for candidate search
    IF candidate search data are not valid
      return error
    ELSE
      IF READ database for candidate search data is successful
        return success
      ELSE
        return error

END
```

1.9.2-ALG filterCandidates() Algorithm

Purpose:

Filters candidates based on certain criteria.

Pseudocode

```
filterCandidates(studentProfiles, candidateCriteria)

    matchedStudents <- EMPTY

    FOR EACH student IN studentProfiles
        IF student.skills == companyCriteria.skills
        AND student.GPA >= companyCriteriaGPA
            ADD student TO matchedStudents

    PUT matchedStudents

END
```

1.9.3-ALG searchJobs() Algorithm

Purpose:

This algorithm manages the job search pipeline by validating criteria, checking for cached results, querying the database index, and filtering and ranking the final job listings for the user.

Pseudocode

```
searchJobs(keywords, selectedFilters)
  filtersAreValid <- validateFilters(selectedFilters)

  IF filtersAreValid == FALSE
    PUT "Invalid search filters applied."
    RETURN emptyList
  ENDIF

  cachedResults <- CALL retrieveResults(keywords, selectedFilters)
  IF cachedResults != NULL
    RETURN cachedResults
  ENDIF

  searchQuery <- CALL buildSearchQuery(keywords, selectedFilters)
  rawResults <- CALL executeSearch(searchQuery)
  filteredResults <- CALL filterResults(rawResults, selectedFilters)
  finalRankedResults <- CALL rankResults(filteredResults)

  CALL storeResults(keywords, selectedFilters, finalRankedResults)

  RETURN finalRankedResults
ENDFUNCTION
```

1.9.5-ALG validateFilters() Algorithm

Purpose:

This algorithm ensures data integrity by verifying that user-selected search filters—such as realistic GPA ranges, positive pay scales, and logical date ranges—contain valid data before processing.

Pseudocode

```
validateFilters(selectedFilters)
  IF selectedFilters == NULL
    RETURN TRUE
  ENDIF

  IF selectedFilters.payScale != NULL
    IF selectedFilters.payScale.min < 0 OR selectedFilters.payScale.max <
      selectedFilters.payScale.min
      RETURN FALSE
    ENDIF
  ENDIF

  IF selectedFilters.gpaRequirement != NULL
    IF selectedFilters.gpaRequirement < 0.0 OR selectedFilters.gpaRequirement >
      4.0
      RETURN FALSE
    ENDIF
  ENDIF

  IF selectedFilters.location != NULL
    IF selectedFilters.location == ""
      RETURN FALSE
    ENDIF
  ENDIF

  IF selectedFilters.internshipDates != NULL
    IF selectedFilters.internshipDates.endDate <
      selectedFilters.internshipDates.startDate
      RETURN FALSE
    ENDIF
  ENDIF

  RETURN ENDFUNCTION
```

1.9.7-ALG buildSearchQuery() Algorithm

Purpose:

buildSearchQuery builds a search query from the user's search terms and filters so the search service can find matching job results.

Pseudocode

```
buildSearchQuery(searchTerms, searchFilters)
  query ← searchTerms

  IF searchFilters is not empty THEN
    FOR EACH filter IN searchFilters
      query ← query + " " + filter
    END FOR
  END IF

  PUT query
END buildSearchQuery
```

1.9.7-ALG processWildcardSearch() Algorithm

Purpose:

The algorithm validates the wildcard search data and if data are valid then it performs the wildcard search according to the search data either for company search or for job search or for candidate search.

Pseudocode

```
processWildcardSearch(wildcardSearchData)

  IF wildcardSearchData for company search
    IF company search data containing wildcard are not valid
      return error
    ELSE
      IF READ database for company search data containing wildcard is successful
        return success
      ELSE
        return error

  IF wildcardSearchData for job search
    IF job search data containing wildcard are not valid
      return error
    ELSE
      IF READ database for job search data containing wildcard is successful
        return success
      ELSE
        return error

  IF wildcardSearchData for candidate search
    IF candidate search data containing wildcard are not valid
      return error
    ELSE
      IF READ database for candidate search data containing wildcard is
        successful
        return success
      ELSE
        return error

END
```


1.9.8-ALG retrieveResults() Algorithm

Purpose:

retrieveResults retrieves the matching search results from the search source or cache and returns them for display.

Pseudocode

```
retrieveResults(query)
    results ← searchIndex(query)

    IF results is empty THEN
        results ← getCacheResults(query)
    END IF

    PUT results
END retrieveResults
```

1.9.12-ALG rankResults() Algorithm

Purpose:

The algorithm validates search results, calculates a relevance score based on criteria such as skills, GPA, location, and user follow status, and returns the highest-ranked results.

Pseudocode

```
rankResults(searchResultsData)

  IF searchResultsData is empty OR searchResultsData is null
    return error
  ELSE
    rankedResultsList <- empty list

    FOR EACH result IN searchResultsData
      relevanceScore <- 0

      IF result matches exact search keyword
        relevanceScore <- relevanceScore + 100
      ELSE
        IF result matches wildcard pattern
          relevanceScore <- relevanceScore + 50
        ELSE
          relevanceScore <- relevanceScore + 0

      IF result type is "Student Profile" OR result type is "Resume"
        IF student matches 50% or more of required skills for the
          job posting
          relevanceScore <- relevanceScore + 80
        ELSE
          relevanceScore <- relevanceScore + 0
        IF student GPA >= company minimum GPA requirement
          relevanceScore <- relevanceScore + 30
        ELSE
          relevanceScore <- relevanceScore + 0

        IF student has relevant Project Experience OR Job
          Experience
          relevanceScore <- relevanceScore + 20
        ELSE
          relevanceScore <- relevanceScore + 0

      IF result type is "Job Posting" OR result type is "Company
        Profile"
        IF student user is following this company
          relevanceScore <- relevanceScore + 60
        ELSE
          relevanceScore <- relevanceScore + 0
```

```
        IF job location matches student preferred location
            relevanceScore <- relevanceScore + 25
        ELSE
            relevanceScore <- relevanceScore + 0

        IF job pay scale matches student expected compensation
            relevanceScore <- relevanceScore + 20
        ELSE
            relevanceScore <- relevanceScore + 0

        result.score <- relevanceScore
        ADD result TO rankedResultsList

    IF SORT rankedResultsList BY result.score DESCENDING is successful
        IF rankedResultsList is NOT empty
            return rankedResultsList
        ELSE
            return error
    ELSE
        return error
```

END

1.10 Communication Services Algorithms

1.10.3-ALG buildNotification() Algorithm

Purpose:

Creates notification content using information contained in the notification job and formats the notification delivery.

Pseudocode

```
buildNotification(notificationJob)

  READ notificationJob

  eventData <- notificationJob.eventData

  IF eventData is missing
    RETURN "Empty Notification"

  formattedMessage <- Notificationformatter.formatNotification(eventData)

  notification.message <- formattedMessage

  notification.title <- eventData.title

  notification.timestamp <- Current Time

  notification.priority <- eventData.priority

  RETURN notification
```

1.10.3-ALG orchestrateNotification() Algorithm

Purpose:

Coordinate the notification workflow by creating the notification, determining recipients, delivering notifications, archiving results, and updating news feed.

Pseudocode

```
orchestrateNotification(notification)

  READ notificationJob

  processingStatus <- "Processing"

  notification <- buildNotification(notificationJob)

  IF notification is empty
    processingStatus <- "Failed"
    RETURN processingStatus

  recipientList <- resolveRecipients(notificationJob)

  IF recipientList is empty
    processingStatus <- "No Recipients"
    archiveNotification(notification)
    RETURN processingStatus

  deliveryStatus <- deliveryNotification(notification, recipientList)

  archiveNotification(notification)

  updateNewsFeed(notification)

  IF deliveryStatus = "Success"
    processingStatus <- "Completed"
  ELSE
    processingStatus <- "partial Failure"

  RETURN processingStatus
```

1.10.3-ALG resolveRecipients() Algorithm

Purpose:

Determines which users should receive a notification by applying recipient rules and removing users who should not receive notifications

Pseudocode

```
resolveRecipients(notificationJob)

  READ notificationJob

  candidateRecipients <- applyRecipientRules(notificationJob)

  recipientList <- Empty List

  FOR EACH currentUser IN candidateRecipients
    IF filterMutedUsers(currentUser) = FALSE
      ADD currentUser TO recipientList

  RETURN recipientList
```

1.10.4-ALG formatNotification() Algorithm

Purpose:

Formats a notification job into notification content and prepares the message content needed for delivery.

Pseudocode

```
formatNotification(notificationJob)

  READ notificationJob
  READ templateData

  notificationContent <- Empty NotificationContent

  notificationContent.title <- getTitle(templateData, notificationJob)
  notificationContent.bodyText <- getBodyText(templateData, notificationJob)
  notificationContent.displayLink <- getDisplayLink(notificationJob)

  IF notificationContent.bodyText is empty
    RETURN Empty NotificationContent

  pushMessage <- formatPushMessage(notificationContent)
  inAppMessage <- formatInAppMessage(notificationContent)

  IF pushMessage is not empty
    ADD pushMessage TO notificationContent

  IF inAppMessage is not empty
    ADD inAppMessage TO notificationContent

  RETURN notificationContent
```

Local Data:

templateData : NotificationTemplate

notificationContent : NotificationContent

pushMessage : NotificationContent

inAppMessage : NotificationContent

1.10.5-ALG saveNotification() Algorithm

Purpose:

Stores notification content for the resolved recipients and creates notification records for later use.

Pseudocode

```
saveNotification(notificationContent, recipientList)

  READ notificationContent
  READ recipientList
  READ notificationDb

  IF notificationContent is empty
    RETURN Empty NotificationRecord

  IF recipientList has no recipients
    RETURN Empty NotificationRecord

  notificationRecord <- Empty NotificationRecord
  notificationRecord.notificationId <- createNotificationId()
  notificationRecord.readStatus <- unread
  notificationRecord.archiveStatus <- active

  FOR EACH recipient IN recipientList
    WRITE notificationContent TO notificationDb FOR recipient
    WRITE notificationRecord TO notificationDb FOR recipient

  RETURN notificationRecord
```

Local Data:

notificationDb : NotificationDb

notificationRecord : NotificationRecord

recipient : Recipient

1.10.6-ALG resolveRecipients() Algorithm

Purpose:

Determines which users should receive a notification by applying recipient rules and removing muted users.

Pseudocode

```
resolveRecipients(notificationJob)

  READ notificationJob
  READ recipientRules
  READ userPreferences

  candidateRecipients <- applyRecipientRule(userPreferences)

  recipientList <- filterMutedUsers(candidateRecipients)

  RETURN recipientList
```

Local Data:

userPreferences : UserPreferences
recipientRules : RecipientRules
candidateRecipients : RecipientList
recipientList : RecipientList

1.10.7-ALG deliverPushNotification() Algorithm

Purpose:

Sends a routed notification to the recipient's device through the push service client. It looks up the device token, attempts delivery with a bounded number of retries, records the outcome in the delivery log, and returns a DeliveryResult describing whether the push succeeded.

Pseudocode

```
deliverPushNotification(notification)

    GET notification

    recipientId <- notification.recipientId
    deviceToken <- tokenRegistry.lookupToken(recipientId)

    IF deviceToken is empty
        result.status <- "FAILED"
        result.reason <- "No registered device token"
        WRITE result TO deliveryLog
        RETURN result

    message <- notification.payload
    attempts <- 0
    delivered <- false

    WHILE attempts < MAX_ATTEMPTS AND delivered is false
        response <- pushServiceClient.send(deviceToken, message)
        attempts <- attempts + 1

        IF response.accepted is true
            delivered <- true
        ELSE
            WAIT briefly before next retry

    IF delivered is true
        result.status <- "DELIVERED"
    ELSE
        result.status <- "FAILED"
        result.reason <- response.errorMessage

    result.attempts <- attempts
    WRITE result TO deliveryLog
    RETURN result
```

Local Data:

deviceToken : Text

message : NotificationPayload
attempts : Number
delivered : Boolean
result : DeliveryResult

1.10.7-ALG routeToService() Algorithm

Purpose:

Selects the correct delivery channel and service target for an incoming routing request and packages the request into a RoutedNotification. If no channel is requested, it chooses push when notifications are available and falls back to in-app delivery otherwise.

Pseudocode

```
routeToService(request)

  GET request

  IF request is empty OR request.payload is empty
    routed.status <- "INVALID"
    RETURN routed

  channel <- request.preferredChannel

  IF channel is not provided
    IF NotificationsAvailable(request.recipientId) is true
      channel <- "PUSH"
    ELSE
      channel <- "IN_APP"

  SWITCH channel
    CASE "PUSH"
      serviceTarget <- pushServiceClient
    CASE "IN_APP"
      serviceTarget <- inAppServiceClient
    DEFAULT
      serviceTarget <- defaultServiceClient

  routed.recipientId <- request.recipientId
  routed.payload <- request.payload
  routed.channel <- channel
  routed.serviceTarget <- serviceTarget
  routed.status <- "ROUTED"

  RETURN routed
```

Local Data:

channel : Text

serviceTarget : ServiceClient
routed : RoutedNotification

1.10.8-ALG retrieveFeedItems() Algorithm

Purpose:

Reads a user's feed items from the feed repository, filters them by the category and unread criteria carried on the request, sorts the results newest-first, applies the requested item limit, and returns the assembled FeedItemList.

Pseudocode

```
retrieveFeedItems(request)

  GET request

  userId <- request.userId

  IF userId is empty
    RETURN empty FeedItemList

  READ allItems FROM feedRepository FOR userId

  filteredItems <- empty list

  FOR each item IN allItems
    IF request.category is provided
      AND item.category is not request.category
        CONTINUE
    IF request.unreadOnly is true AND item.hasBeenRead is true
      CONTINUE
    ADD item TO filteredItems

  SORT filteredItems BY item.timestamp DESCENDING

  IF request.maxItems > 0
    AND size(filteredItems) > request.maxItems
      filteredItems <- first request.maxItems items OF filteredItems

  feedItemList.items <- filteredItems
  feedItemList.count <- size(filteredItems)

  RETURN feedItemList
```

Local Data:

allItems : List of FeedItem
filteredItems : List of FeedItem
feedItemList : FeedItemList

1.11 Recommendation Service Algorithms

1.11.2-ALG applyCandidateFilters() Algorithm

Purpose:

Filters searchable student profiles by comparing each candidate against the employer's selected search criteria and returning only candidates who satisfy the required filters.

Pseudocode

```
applyCandidateFilters(candidateList, normalizedSearchCriteria)

GET candidateList
GET normalizedSearchCriteria

filteredCandidates ← createEmptyCandidateList()

FOR each candidate IN candidateList

    IF normalizedSearchCriteria.requiredSkills is not empty AND candidate does
      not match requiredSkills
      CONTINUE to next candidate
    END IF

    IF normalizedSearchCriteria.degreeProgram is not empty AND candidate does not
      match degreeProgram
      CONTINUE to next candidate
    END IF

    IF normalizedSearchCriteria.locationPreference is not empty AND candidate
      does not match locationPreference
      CONTINUE to next candidate
    END IF

    IF normalizedSearchCriteria.minimumGpa is not empty AND candidate.gpa is less
      than normalizedSearchCriteria.minimumGpa
      CONTINUE to next candidate
    END IF

    IF normalizedSearchCriteria.experienceLevel is not empty AND candidate does
      not match experienceLevel
      CONTINUE to next candidate
    END IF

    IF normalizedSearchCriteria.availability is not empty AND candidate does not
      match availability
      CONTINUE to next candidate
    END IF

    PUT candidate INTO filteredCandidates

END FOR

RETURN filteredCandidates
```

Local Data:

filteredCandidates : List of StudentProfile

candidate : StudentProfile

1.11.2-ALG recommendCandidates() Algorithm

Purpose:

Recommends qualified student candidates to an employer by validating the candidate search request, applying employer-selected filters, scoring eligible candidates, ranking the results, and returning a ranked recommendation list.

Pseudocode

```
recommendCandidates(employerSearchQuery)

GET employerSearchQuery

validationErrors ← validateEmployerSearchQuery(employerSearchQuery)

IF validationErrors is not empty
    candidateRankedList ← createEmptyCandidateRankedList()
    PUT validationErrors INTO candidateRankedList
    RETURN candidateRankedList
END IF

normalizedSearchCriteria ← normalizeSearchCriteria(employerSearchQuery)

candidateList ← READ searchable student profiles from ResumeIndex

filteredCandidates ← applyCandidateFilters(candidateList,
    normalizedSearchCriteria)

candidateRankedList ← createEmptyCandidateRankedList()

FOR each candidate IN filteredCandidates
    matchScore ← calculateCandidateScore(candidate,
        normalizedSearchCriteria)

    IF normalizedSearchCriteria.minimumMatchScore is empty OR matchScore >=
        normalizedSearchCriteria.minimumMatchScore
        recommendation ← createCandidateRecommendation(candidate,
            matchScore)
        PUT recommendation INTO candidateRankedList
    END IF
END FOR

candidateRankedList ← rankCandidateResults(candidateRankedList)

IF normalizedSearchCriteria.resultLimit is not empty
    candidateRankedList ← limitCandidateResults(candidateRankedList,
        normalizedSearchCriteria.resultLimit)
END IF

RETURN candidateRankedList
```

Local Data:

validationErrors : List of String

normalizedSearchCriteria : EmployerSearchQuery

candidateList : List of StudentProfile

filteredCandidates : List of StudentProfile

candidateRankedList : CandidateRankedList

matchScore : Number

recommendation : CandidateRecommendation

1.11.2-ALG calculateCandidateScore() Algorithm

Purpose:

Calculates a candidate ranking score for an employer search or recommendation request by evaluating required filters, skills, experience, profile and resume information, and weighted criteria; returns a bounded score, a score breakdown, and the candidate eligibility status.

Pseudocode

```
calculateCandidateScore(  
    candidateProfile, candidateResume, employerSearchCriteria,  
    requiredSkills, requiredExperience, candidateFilters,  
    scoringWeights)  
  
READ candidateProfile and candidateResume  
READ employerSearchCriteria and candidateFilters  
READ requiredSkills, requiredExperience, and scoringWeights  
  
candidateScore ← 0  
maximumScore ← 100  
scoreBreakdown ← empty score breakdown  
candidateEligibilityStatus ← "Eligible"  
  
IF required profile or resume information is missing  
    candidateEligibilityStatus ← "Incomplete"  
    missingInformationPenalty ← maximumScore  
    scoreBreakdown ← RECORD missingInformationPenalty  
    WRITE candidateScore, scoreBreakdown, candidateEligibilityStatus  
    RETURN candidateScore, scoreBreakdown, candidateEligibilityStatus  
END IF  
  
IF candidate does not satisfy required candidateFilters  
    candidateEligibilityStatus ← "Filtered Out"  
    scoreBreakdown ← RECORD "Required filters not satisfied"  
    WRITE candidateScore, scoreBreakdown, candidateEligibilityStatus  
    RETURN candidateScore, scoreBreakdown, candidateEligibilityStatus  
END IF  
  
skillMatch ← compareSkills(candidateProfile.skills, requiredSkills)  
experienceMatch ← compareExperience(candidateResume.experience,  
    requiredExperience)  
profileMatch ← evaluateProfileInformation(candidateProfile,  
    candidateResume)  
filterMatch ← evaluatePreferredFilters(candidateProfile,  
    employerSearchCriteria)  
  
skillScore ← SCALE skillMatch BY scoringWeights.skills  
experienceScore ← SCALE experienceMatch BY scoringWeights.experience  
profileScore ← SCALE profileMatch BY scoringWeights.profile  
filterScore ← SCALE filterMatch BY scoringWeights.filters  
  
missingInformationPenalty ← calculateMissingInformationPenalty(  
    candidateProfile, candidateResume)  
candidateScore ← skillScore + experienceScore + profileScore  
candidateScore ← candidateScore + filterScore  
candidateScore ← candidateScore - missingInformationPenalty  
scoreBreakdown ← RECORD skillScore, experienceScore, profileScore,  
    filterScore, missingInformationPenalty  
  
IF candidateScore > maximumScore  
    candidateScore ← maximumScore  
END IF  
IF candidateScore < 0  
    candidateScore ← 0  
END IF  
  
WRITE candidateScore, scoreBreakdown, candidateEligibilityStatus  
RETURN candidateScore, scoreBreakdown, candidateEligibilityStatus
```

Local Data:

candidateScore : Number
maximumScore : Number
skillMatch : Number
experienceMatch : Number
profileMatch : Number
filterMatch : Number
skillScore : Number
experienceScore : Number
profileScore : Number
filterScore : Number
missingInformationPenalty : Number
scoreBreakdown : ScoreBreakdown
candidateEligibilityStatus : EligibilityStatus

1.11.2-ALG rankMatches() Algorithm

Purpose:

Accepts a list of Recommendation objects and returns them sorted in descending order by matchScore, so the highest-scoring candidates appear first. The algorithm uses an insertion sort, which is efficient for small to medium lists and preserves the relative order of equal scores (stable sort). If the input list is empty, it is returned immediately.

Pseudocode

```
rankMatches(matchList)

// Sorts a list of Recommendation objects in descending order
// by their matchScore and returns the ranked list

INPUT:
    matchList: List<Recommendation>
OUTPUT
    rankedList: List<Recommendation>

BEGIN
    IF SIZE(matchList) = 0 THEN
        RETURN matchList
    ENDIF

    n <- SIZE(matchList)
    rankedList <- matchList

    // Insertion sort descending by matchScore
    i <- 1
    WHILE i < n DO
        current <- rankedList[i]
        j <- i - 1

        WHILE j >= 0 AND rankedList[j].matchScore < current.matchScore DO
            rankedList[j + 1] <- rankedList[j]
            j <- j - 1
        END WHILE

        rankedList[j + 1] <- current
        i <- i + 1
    END WHILE

    RETURN rankedList
END
```

Local Data:

n: Integer

rankedList: List<Recommendation>

i: Integer

j: Integer

current: Recommendation

1.11.3-ALG compareExperience() Algorithm

Purpose:

Compares a student's years of work experience against the experience requirement of a job posting and returns a weighted score out of 30. If the student meets or exceeds the requirement, they receive full marks. If not, a proportional score is calculated using the ratio of student years to require years. A small bonus is awarded for exceeding the requirement by two or more years, clamped to a maximum of 30 points.

Pseudocode

```
compareExperience(studentProfile, jobPosting)

// Compares a student's work experience against job requirements
// and returns a weighted score out of 30

INPUT:
    studentProfile : StudentProfile
    jobPosting : JobPosting

OUTPUT
    experienceScore: Integer

BEGIN
    requiredYears <- jobPosting.requiredExperience
    studentYears <- studentProfile.yearsExperience
    experienceScore <- 0

    IF requireYears = 0 THEN
        experienceScore <- 30
    ELSE IF studentYears >= requiredYears THEN
        experienceScore <- 30
    ELSE
        experienceRatio <- studentYears / requiredYears
        experienceScore <- FLOOR(experienceRatio * 30)
    END IF

    // Award bonus point for exceeding requirement by 2+ years
    IF studentYears >= requiredYears + 2 THEN
        bonusPoints <- studentYears - requiredYears
        experienceScore <- experienceScore + FLOOR(bonusPoints / 2)
    ENDIF

    // Clamp to maximum of 30
    IF experienceScore > 30 THEN
        experienceScore <- 30
    END IF

    RETURN experienceScore

END
```

Local Data:

requiredYears: Integer

studentYears: Integer

experienceRation: Float
bonusPoints: Integer
experienceScore: Integer

1.11.3-ALG calculateMatchScore() Algorithm

Purpose:

Computes 0-100 composite score across four weighted sub-scores: skill match (40pts, delegated to compareSkills()), experience (30pts), education (20 pts), and location/remote (10 pts). Score is clamped at 100 before returning.

Pseudocode

```
calculateMatchScore(studentProfile, jobPosting)
// Returns a numeric match score (0-100) between a student and a job posting

INPUT:
    studentProfile : StudentProfile
    jobPosting : JobPosting
OUTPUT
    matchScore : Integer

BEGIN
    skillScore <- compareSkills(studentProfile.skills, jobPosting.requiredSkills)
    experienceScore <- 0
    educationScore <- 0
    locationScore <- 0

    // Score experience level
    IF studentProfile.yearsExperience >= jobPosting.requiredExperience THEN
        experienceScore <- 30
    ELSE
        experienceScore <- FLOOR((studentProfile.yearsExperience /
jobPosting.requiredExperience) * 30
    ENDIF

    // Score education level
    IF studentProfile.educationLevel >= jobPosting.requiredEducation THEN
        educationScore <- 20
    ELSE
        education <- 10
    ENDIF

    // Score location preference
    IF studentProfile.location = jobPosting.location THEN
        locationScore <- 10
    ELSE IF jobPosting.isRemote = TRUE THEN
        locationScore <- 10
    ELSE
        locationScore <- 0
    END IF
    matchScore <- skillScore + experienceScore + educationScore + locationScore

    // Clamp to 0-100 range
    IF matchScore > 100 THEN
        matchScore <- 100
    END IF

    RETURN matchScore
END
```

Local Data:

skillScore: Integer

experienceScore: Integer

educationScore: Integer
locationScore: Integer
matchScore: Integer

1.11.3-ALG compareSkills() Algorithm

Purpose:

Iterates every required skill against the student's skill list using a case-insensitive name match, then scales the match ratio to 40 points. Called as a sub-routine by Algorithm 1.

Pseudocode

```
compareSkills(studentSkills, requiredSkills)
// Compares students skills against job required skills and returns
// a weighted score out of 40 based on match skill count

INPUT:
    studentSkills : List<Skill>
    requiredSkills: List<Skill>
OUTPUT
    skillScore : Integer

BEGIN
    matchCount <- 0
    totalRequired <- SIZE(requiredSkills)

    IF totalRequired = 0 THEN
        RETURN 0
    ENDIF

    FOR EACH requiredSkill IN requiredSkills DO
        FOR EACH studentSkill IN studentSkills DO
            IF LOWERCASE(studentSkill.name)
= LOWERCASE(requiredSkill.name) THEN
                matchCount <- matchCount + 1
                BREAK
            ENDIF
        ENDFOR
    ENDFOR

    matchRatio <- matchCount / totalRequired
    skillScore <- FLOOR(matchRatio * 40)

    RETURN skillScore

END
```

Local Data:

matchCount: Integer

totalRequired: Integer
matchRatio: Float
skillScore: Integer
requiredSkill: Skill
studentSkill: Skill

1.11.5-ALG generateRecommendations() Algorithm

Purpose:

Coordinates the final candidate recommendation process by validating an employer request, retrieving candidate and resume records, applying filters, scoring eligible candidates, ranking matches, limiting results, and returning the recommendation list and summary.

Pseudocode

```
generateRecommendations(  
    employerRecommendationRequest, resumeIndex,  
    recommendationLimit, scoringWeights)  
  
READ employerRecommendationRequest  
READ employerSearchCriteria and candidateFilters from request  
READ recommendationLimit and scoringWeights  
  
recommendationList ← empty recommendation list  
scoredCandidateList ← empty scored candidate list  
recommendationSummary ← empty recommendation summary  
noResultsMessage ← empty string  
  
IF employerRecommendationRequest is missing required criteria  
    noResultsMessage ← "Recommendation request is incomplete"  
    recommendationSummary ← RECORD noResultsMessage  
    WRITE noResultsMessage, recommendationSummary  
    RETURN recommendationList, empty ranked list,  
        recommendationSummary  
END IF  
  
candidateProfileList ← READ matching candidate and resume records  
    from resumeIndex using employerSearchCriteria  
  
IF candidateProfileList is empty  
    noResultsMessage ← "No candidates found"  
    recommendationSummary ← RECORD noResultsMessage  
    WRITE noResultsMessage, recommendationSummary  
    RETURN recommendationList, empty ranked list,  
        recommendationSummary  
END IF  
  
filteredCandidateList ← applyCandidateFilters(  
    candidateProfileList, candidateFilters)  
  
FOR each candidateRecord IN filteredCandidateList  
    candidateScoreResult ← calculateCandidateScore(  
        candidateRecord.profile, candidateRecord.resume,  
        employerSearchCriteria, employerSearchCriteria.requiredSkills,  
        employerSearchCriteria.requiredExperience, candidateFilters,  
        scoringWeights)  
    IF candidateScoreResult.candidateEligibilityStatus is "Eligible"  
        IF candidateScoreResult.candidateScore >=  
            employerSearchCriteria.minimumRecommendationScore  
            candidateRecommendation ← CREATE recommendation from  
                candidateRecord and candidateScoreResult  
            PUT candidateRecommendation INTO scoredCandidateList  
        END IF  
    END IF  
END FOR  
  
IF scoredCandidateList is empty  
    noResultsMessage ← "No qualified recommendations found"  
    recommendationSummary ← RECORD noResultsMessage  
    WRITE noResultsMessage, recommendationSummary  
    RETURN recommendationList, empty ranked list,  
        recommendationSummary  
END IF  
  
rankedCandidateList ← rankMatches(scoredCandidateList)  
  
FOR each rankedCandidate IN rankedCandidateList  
    IF count recommendationList < recommendationLimit  
        PUT rankedCandidate INTO recommendationList  
    END IF  
END FOR  
  
recommendationSummary ← CREATE summary from recommendationList  
WRITE recommendationList, rankedCandidateList, recommendationSummary  
RETURN recommendationList, rankedCandidateList, recommendationSummary
```

Local Data:

recommendationList : List of CandidateRecommendation
scoredCandidateList : List of CandidateRecommendation
rankedCandidateList : List of CandidateRecommendation
recommendationSummary : RecommendationSummary
noResultsMessage : String
candidateProfileList : List of CandidateRecord
filteredCandidateList : List of CandidateRecord
candidateRecord : CandidateRecord
candidateScoreResult : CandidateScoreResult
candidateRecommendation : CandidateRecommendation

1.12 External Services Algorithms

1.12.1-ALG getAddress()

Purpose:

Retrieves the complete address entered by the user and prepares it for validation.

Pseudocode

```
FUNCTION getAddress ()  
  
    GET streetAddress  
    GET suiteNumber  
    GET city  
    GET state  
    GET zipCode  
    GET country  
  
    RETURN AddressRequest  
  
END FUNCTION
```

1.12.1-ALG validateAddress()

Purpose:

Verifies that all required address information is present before attempting USPS validation.

Pseudocode

```
FUNCTION validateAddress

    IF streetAddress IS EMPTY OR
       City IS EMPTY OR
       STATE IS EMPTY OR
       zipCode IS EMPTY THEN

        RETURN FALSE

    ENDIF
    RETURN TRUE

END FUNCTION
```

1.12.1-ALG standardizeAddress()

Purpose:

Formats the validated address according to USPS standards and returns the standardized address.

Pseudocode

```
FUNCTION standardizeAddress (AddressRequest)

    IF validateAddress (AddressRequest) = FALSE THEN
        RETURN FALSE
    ENDIF

    READ USPS Standard

    standardizedAddress <- Format AddressRequest using USPS Standard

    RETURN standardizedAddress

END FUNCTION
```

1.12.1-ALG getValidatedAddress() Algorithm

Purpose:

Retrieves a USPS-validated address by validating the city and ZIP code and returning a standardized address when the input is valid.

Pseudocode

```
FUNCTION getValidatedAddress (addressRequest)
    GET streetAddress
    GET suiteNumber
    GET city GET state
    GET zipCode
    GET country

    IF validateCity(city, state) = FALSE THEN
        WRITE "Invalid City"
        RETURN NULL
    ENDIF

    IF validateZip(city, state, zipCode) = FALSE THEN
        WRITE "Invalid ZIP Code"
        RETURN NULL
    ENDIF

    PUT addressRequest INTO ValidatedAddress

    RETURN ValidatedAddress
END FUNCTION
```

1.12.1-ALG validateCity() Algorithm

Purpose:

Verifies that the provided city exists and is valid for the specified state according to USPS address standards.

Pseudocode

```
FUNCTION validateCity(city, state)
  GET city
  GET state

  IF city IS EMPTY OR state IS EMPTY THEN
    RETURN FALSE
  ENDIF
  READ USPS City Listing
  IF city EXISTS FOR state THEN
    RETURN TRUE
  ELSE
    RETURN FALSE
  ENDIF
END FUNCTION
```

1.12.1-ALG validateZip() Algorithm

Purpose:

Verifies that the provided ZIP code corresponds to the specified city and state according to USPS address standards.

Pseudocode

```
FUNCTION validateZip(city, state, zipCode)

    GET city GET state
    GET zipCode

    IF zipCode IS EMPTY THEN
        RETURN FALSE
    ENDIF

    READ USPS ZIP Code Listing

    IF zipCode MATCHES city AND state THEN
        RETURN TRUE
    ELSE
        RETURN FALSE
    ENDIF
END FUNCTION
```

1.12.3-ALG pushNotificationGateway() Algorithm

Purpose:

This algorithm manages the delivery of push notifications to a user's registered devices. It validates that device tokens are available, sends the notification message to each device, tracks successful and failed deliveries, and returns a summary of the notification request, including delivery status and any errors encountered.

Pseudocode

```
pushNotificationGateway()

GET userId
GET deviceTokens
GET messageTitle
GET messageBody
GET validatedAddress
GET autocompleteContext

notificationID <- Generate Notification Identifier

successfulDevices <- 0
failedDevices <- 0
errorList <- Empty List

IF deviceTokens is empty THEN
    WRITE status <- "FAILED"
    WRITE errorList <- "No device tokens provided"

    RETURN notificationId,
        status,
        Current Timestamp,
        successfulDevices,
        failedDevices,
        errorList
END IF

FOR EACH deviceToken IN deviceTokens
    deliveryStatus <- Send Push Notification(
        deviceToken,
        messageTitle,
        messageBody,
        validatedAddress,
        autocompleteContext)
```

```

    IF deliveryStatus = SUCCESS THEN
        successfulDevices <- successfulDevices + 1
    ELSE
        failedDevices <- failedDevices + 1
        ADD deliveryStatus error TO errorList
    END IF
END FOR

IF successfulDevices > 0 THEN
    WRITE status <- "SENT"
ELSE
    WRITE status <- "FAILED"
END IF

RETURN notificationId,
    status,
    Current Timestamp,
    successfulDevices,
    failedDevices,
    errorList

```

Local Data:

notificationId : Notification Identifier
 successfulDevices : Number
 failedDevices : Number
 deliveryStatus : Status
 errorList : List of Error Messages

1.13 Analytics Service Algorithms

1.13.1-ALG getResumeSubmissions() Algorithm

Purpose:

Fetches all resume form submissions for a job posting and accepts parameters to sort and filter candidates.

Pseudocode

```
getResumeSubmissions()

  PUT "Welcome to Resume Submissions"

  SET jobPosting ← ""

  WHILE jobPosting != "0"

    PUT "Please enter a job posting to see resume submissions for that job
    posting or enter 0 to exit."

    GET jobPosting
    SET jobPostingId ← search JobPosting table WHERE name = jobPosting

    IF jobPosting == "0"
      PUT "Exiting Resume Submissions."
    ELSE IF jobPostingId exists
      PUT "Select filters or leave blank for default."
      GET SortCandidates

      SET ResumeSubmissions ← fetch by jobPostingId and SortCandidates
      PUT DisplayResumeSubmissions(ResumeSubmissions)
    ELSE
      PUT "Invalid entry. The job posting entered does not exist."

  END WHILE

END
```

1.13.1-ALG manageJobPosting() Algorithm

Purpose:

Used so a company representative can add, edit, or delete a job posting.

Pseudocode

```
manageJobPosting()

  SET userChoice ← ""

  PUT "Welcome to the Job Posting Manager!"

  WHILE userChoice != 4

    PUT "Please select a choice: [1] Add New Job Posting, [2] Edit a Job Posting, [3] Delete a Job Posting, [4] Exit Job Posting Manager"

    GET userChoice

    IF userChoice == 1
      GET jobPostingData
      addJobPosting(jobPostingData)
    ELSE IF userChoice == 2
      GET jobPostingId
      GET jobPostingData
      editJobPosting(jobPostingId, jobPostingData)
    ELSE IF userChoice == 3
      GET jobPostingId
      deleteJobPosting(jobPostingId)
    ELSE IF userChoice == 4
      PUT "Exiting Job Posting Manager"
    ELSE
      PUT "Invalid option. Please enter a valid number."

  END WHILE

END
```

1.13.2-ALG displayFilteredCandidates() Algorithm

Purpose:

Displays filtered candidate profiles and their resumes within 3 seconds of search request submission.

Pseudocode

```
displayFilteredCandidates()  
  PUT "Welcome to Candidate Search Manager!"  
  
  SET exitCandidateSearchManager - FALSE  
  
  WHILE exitCandidateSearchManager == FALSE  
    PUT "Please check the filters to search by and then click the Search button  
    or click the Exit button to exit: skills, location, work preference,  
    willingness to relocate, position type, start date availability, years of  
    experience, degree level, graduation date, application source, GPA  
  
    GET userAction  
  
    IF userAction == Exit button was clicked  
      SET exitCandidateSearchManager - TRUE  
      PUT "Exiting Candidate Search Manager"  
    ELSE IF userAction == Search button was clicked  
      SET activeSearchFilters - read checked boxes  
  
      SET candidateSearchResults - searchCandidates(activeSearchFilters)  
  
      IF candidateSearchResults exists  
        PUT candidateSearchResults  
      ELSE  
        PUT "There are no candidates for the selected filters."  
  
  END WHILE  
  
END
```

1.13.2-ALG parseWildcardSearches() Algorithm

Purpose:

This algorithm parses the searchQuery to make the parsedQuery.

Pseudocode

```
private parseWildcardSearches()  
  
    GET searchQuery FROM CandidateSearchManager  
  
    CREATE empty parsedResult  
  
    FOR EACH character IN searchQuery:  
        IF character = wildcard symbol  
            APPEND wildcard search pattern TO parsedResult  
        ELSE  
            APPEND character TO parsedResult  
    SET this.parsedQuery = parsedResult
```

Local Data:

parsedResult : String

1.13.2-ALG autoFillSearchBar() Algorithm

Purpose:

This algorithm takes the input that the user has typed up and displays a complete suggestion.

Pseudocode

```
public autoFillSearchBar()  
  
    GET searchInput FROM CandidateSearchManager  
    GET autoFillSuggestions FROM candidateSearchManager  
  
    CREATE empty matchingResults  
  
    FOR EACH suggestion IN autoFillSuggestions  
        IF suggestion CONTAINS searchInput  
            APPEND suggestion CONTAINS TO matchingResults  
  
    PUT matchingResults TO search bar display
```

Local Data:

matchingResults : List<string>

1.13.3-ALG matchResume() Algorithm

Purpose:

This algorithm evaluates whether a student's resume meets a set of required job skills. It retrieves the specified resume, compares the skills listed in the resume against the required skills, calculates the number of matches, and returns a result indicating whether the resume satisfies all required criteria.

Pseudocode

```
matchResume()

GET resumeId
GET requiredSkills

resumeData ← Retrieve Resume(resumeId)

IF resumeData does not exist THEN
    RETURN FALSE
END IF

matchedSkills ← 0

FOR EACH skill IN requiredSkills
    IF skill EXISTS IN resumeData skills THEN
        matchedSkills ← matchedSkills + 1
    END IF
END FOR

IF matchedSkills = LENGTH(requiredSkills) THEN
    matchResult ← TRUE
ELSE
    matchResult ← FALSE
END IF

RETURN matchResult
```

Local Data:

resumeData : Resume

requiredSkills : List of Skills

matchedSkills : Number

matchResult : Boolean

1.13.4-ALG notificationAndNewsFeedbackAnalytics() Algorithm

Purpose:

This algorithm processes user interaction events related to notifications and news content for analytics purposes. It validates incoming event data, records the interaction in the analytics repository, determines whether processing was successful, and returns the status of the operation along with any validation or processing errors.

Pseudocode

```
notificationAndNewsFeedbackAnalytics()

GET eventId
GET userId
GET eventType
GET timestamp
GET sessionId
GET metadata

validationErrors ← Empty List

IF eventId is missing THEN
    ADD "Missing Event ID" to validationErrors
END IF

IF userId is missing THEN
    ADD "Missing User ID" TO validationErrors
END IF

IF eventType is missing THEN
    ADD "Missing Event Type" TO validationErrors
END IF

IF validationErrors is not empty THEN
    RETURN eventId,
        "FAILED",
        Current Timestamp,
        NULL,
        validationErrors
END IF

analyticsId ← Store Analytics Event(
    eventId,
    userId,
    eventType,
    timestamp,
    sessionId,
    metadata)
```

```

IF analyticsId exists THEN
    processingStatus ← "SUCCESS"
ELSE
    processingStatus ← "FAILED"
END IF

RETURN eventId,
    processingStatus,
    Current Timestamp,
    analyticsId,
    validationErrors

```

Local Data:

analyticsId : Analytics Identifier

processingStatus : Status

validationErrors : List of Error Messages

1.13.5-ALG PerformanceMonitor() Algorithm

Purpose:

This algorithm reads activity data, analyzes it, and creates a report or an alert for it.

Pseudocode

```

public monitorPerformance()

    GET activityDataList FROM ActivityData

    CREATE empty monitoringOutputList

    FOR EACH activityData IN activityDataList
        GET eventType FROM activityData.getEventType()

        IF eventType = alert condition
            CREATE new PerformanceAlert
            APPEND PerformanceAlert TO monitoringOutputList
        ELSE
            CREATE new AnalyticsReport
            APPEND AnalyticsReport TO monitoringOutputList

    SET this.monitoringOutputList = monitoringOutputList

```

Local Data:

monitoringOutputList : List<MonitoringOutput>

activityData : ActivityData

eventType : String